

Project Title:

Sentiment Analysis of Tweets, User Input and Visualization of Emotion Distribution

Name: Rushikesh Walmik Raut

Class: TE(AI&DS)

Roll_No: 2337075

Batch: D

Sub: Data Science

Step 1: Import Required Libraries

```
In [9]: import string
import pandas as pd
import matplotlib.pyplot as plt
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from nltk.sentiment.vader import SentimentIntensityAnalyzer
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import make_pipeline
from sklearn.metrics import classification_report
from sklearn.svm import SVC
import chardet
```

```
In [10]: # Download NLTK resources
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('vader_lexicon')
```

```
[nltk_data] Downloading package punkt to  
[nltk_data] C:\Users\Rushikesh\AppData\Roaming\nltk_data...  
[nltk_data] Package punkt is already up-to-date!  
[nltk_data] Downloading package stopwords to  
[nltk_data] C:\Users\Rushikesh\AppData\Roaming\nltk_data...  
[nltk_data] Package stopwords is already up-to-date!  
[nltk_data] Downloading package wordnet to  
[nltk_data] C:\Users\Rushikesh\AppData\Roaming\nltk_data...  
[nltk_data] Package wordnet is already up-to-date!  
[nltk_data] Downloading package vader_lexicon to  
[nltk_data] C:\Users\Rushikesh\AppData\Roaming\nltk_data...  
[nltk_data] Package vader_lexicon is already up-to-date!
```

Out[10]: True

Step 2: Preprocessing Text Function

```
In [11]: def preprocess_text(text):  
# Convert text to lowercase  
lower_text = text.lower()  
  
cleaned_text = lower_text.translate(str.maketrans('', '', string.punctuation))  
  
# Tokenize the text  
tokenized_words = word_tokenize(cleaned_text, "english")  
  
# Remove stopwords  
filtered_words = []  
for word in tokenized_words:  
    if word not in stopwords.words('english'):  
        filtered_words.append(word)  
  
# Lemmatize words  
lemmatized_words = []  
for word in filtered_words:  
    word = WordNetLemmatizer().lemmatize(word)  
    lemmatized_words.append(word)  
  
# Join the lemmatized words back into a string  
preprocessed_text = ' '.join(lemmatized_words)  
return preprocessed_text
```

Step 3: Preprocessing and Loading Data

```
In [12]: # Step 2: Load dataset and train classifier  
with open('test.csv', 'rb') as file:  
    encoding = chardet.detect(file.read())['encoding']  
  
df = pd.read_csv('test.csv', encoding=encoding)  
df = df.dropna()  
df.head()
```

Out[12]:

	textID	text	sentiment	Time of Tweet	Age of User	Country	Populati-20
0	f87dea47db	Last session of the day http://twitpic.com/67ezh	neutral	morning	0-20	Afghanistan	3892834
1	96d74cb729	Shanghai is also really exciting (precisely -...	positive	noon	21-30	Albania	287779
2	eee518ae67	Recession hit Veronique Branquinho, she has to...	negative	night	31-45	Algeria	4385104
3	01082688c6	happy bday!	positive	morning	46-60	Andorra	7726
4	33987a8ee5	http://twitpic.com/4w75p - I like it!!	positive	noon	60-70	Angola	3286627

In [13]: `df.size`

Out[13]: 31806

In [14]: `df.shape`

Out[14]: (3534, 9)

In [15]: `df['sentiment'].value_counts()`

Out[15]: sentiment
 neutral 1430
 positive 1103
 negative 1001
 Name: count, dtype: int64

In [16]: `df = df.drop(df.columns[[0,3,4,5,6,7,8]],axis = 1)`
`df['preprocessed_text'] = df['text'].apply(preprocess_text)`
`df.head()`

Out[16]:

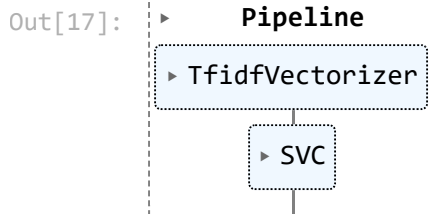
	text	sentiment	preprocessed_text
0	Last session of the day http://twitpic.com/67ezh	neutral	last session day httptwitpiccom67ezh
1	Shanghai is also really exciting (precisely -...	positive	shanghai also really exciting precisely skyscr...
2	Recession hit Veronique Branquinho, she has to...	negative	recession hit veronique branquinho quit compan...
3	happy bday!	positive	happy bday
4	http://twitpic.com/4w75p - I like it!!	positive	httptwitpiccom4w75p like

Step 3: Train SVM Classifier

```
In [17]: X_train, X_test, y_train, y_test = train_test_split(df['preprocessed_text'], df[

svm_classifier = make_pipeline(TfidfVectorizer(), SVC(kernel='linear'))

svm_classifier.fit(X_train, y_train)
```



Step 4: Testing of SVM Classifier

```
In [18]: X_all = df['preprocessed_text']
y_pred_svm = svm_classifier.predict(X_all)

print("Predictions using SVM classifier:\n")
y_pred_svm
```

Predictions using SVM classifier:

```
Out[18]: array(['neutral', 'positive', 'neutral', ..., 'neutral', 'positive',
               'positive'], dtype=object)
```

Step 5: Predictions using Vader SentimentIntensityAnalyzer()

```
In [19]: sid = SentimentIntensityAnalyzer()
y_pred_vader_all = []

for text in X_all:
    sentiment_scores = sid.polarity_scores(text)
    if sentiment_scores['compound'] >= 0.05:
        y_pred_vader_all.append('positive')
    elif sentiment_scores['compound'] <= -0.05:
        y_pred_vader_all.append('negative')
    else:
        y_pred_vader_all.append('neutral')

positive_scores = []
negative_scores = []
neutral_scores = []

for text in X_all:
    sentiment_scores = sid.polarity_scores(text)
    positive_scores.append(sentiment_scores['pos'])
    negative_scores.append(sentiment_scores['neg'])
    neutral_scores.append(sentiment_scores['neu'])

print("Predictions using vader:\n")
y_pred_vader_all[0:10]
```

Predictions using vader:

```
Out[19]: ['neutral',  
          'positive',  
          'negative',  
          'positive',  
          'positive',  
          'positive',  
          'negative',  
          'positive',  
          'neutral',  
          'positive']
```

Step 6: Create DataFrame to store results

```
In [20]: results_df = pd.DataFrame({  
          'Original Text': df['text'],  
          'Preprocessed Text': df['preprocessed_text'],  
          'Positive Polarity Score': positive_scores,  
          'Negative Polarity Score': negative_scores,  
          'Neutral Polarity Score': neutral_scores,  
          'SVM Predictions': y_pred_svm,  
          'Sentiment Analyzer Predictions': y_pred_vader_all  
        })
```

Step 7: Save Results to CSV

```
In [21]: #Save DataFrame to CSV file  
results_df.to_csv('predictions.csv')  
print("CSV file containing predictions saved successfully.")
```

CSV file containing predictions saved successfully.

```
In [22]: result = pd.read_csv('predictions.csv')  
result.head()
```

Out[22]:

	Unnamed: 0	Original Text	Preprocessed Text	Positive Polarity Score	Negative Polarity Score	Neutral Polarity Score
0	0	Last session of the day http://twitpic.com/67ezh	last session day httptwitpiccom67ezh	0.000	0.000	1.000
1	1	Shanghai is also really exciting (precisely -...	shanghai also really exciting precisely skyscr...	0.390	0.000	0.610
2	2	Recession hit Veronique Branquinho, she has to...	recession hit veronique branquinho quit compan...	0.000	0.541	0.459
3	3	happy bday!	happy bday	0.787	0.000	0.213
4	4	http://twitpic.com/4w75p - I like it!!	httptwitpiccom4w75p like	0.714	0.000	0.286

In [23]: `result['SVM Predictions'].value_counts()`

Out[23]: SVM Predictions
 neutral 1598
 positive 1038
 negative 898
 Name: count, dtype: int64

In [25]: `result['Sentiment Analyzer Predictions'].value_counts()`

Out[25]: Sentiment Analyzer Predictions
 positive 1724
 neutral 938
 negative 872
 Name: count, dtype: int64

Step 8: Evaluate Models

Classification report for SVM classifier

```
In [46]: from sklearn.metrics import classification_report

y_true = df['sentiment']

svm_classification_report = classification_report(y_true, y_pred_svm)
print("\nClassification Report for SVM Classifier:\n", svm_classification_report)
```

Classification Report for SVM Classifier:

	precision	recall	f1-score	support
negative	0.91	0.81	0.86	1001
neutral	0.81	0.91	0.86	1430
positive	0.91	0.85	0.88	1103
accuracy			0.86	3534
macro avg	0.88	0.86	0.86	3534
weighted avg	0.87	0.86	0.86	3534

Classification report for sentiment analyzer (VADER)

```
In [47]: vader_classification_report = classification_report(y_true, y_pred_vader_all)
print("\nClassification Report for Sentiment Analyzer (VADER):\n", vader_classif
```

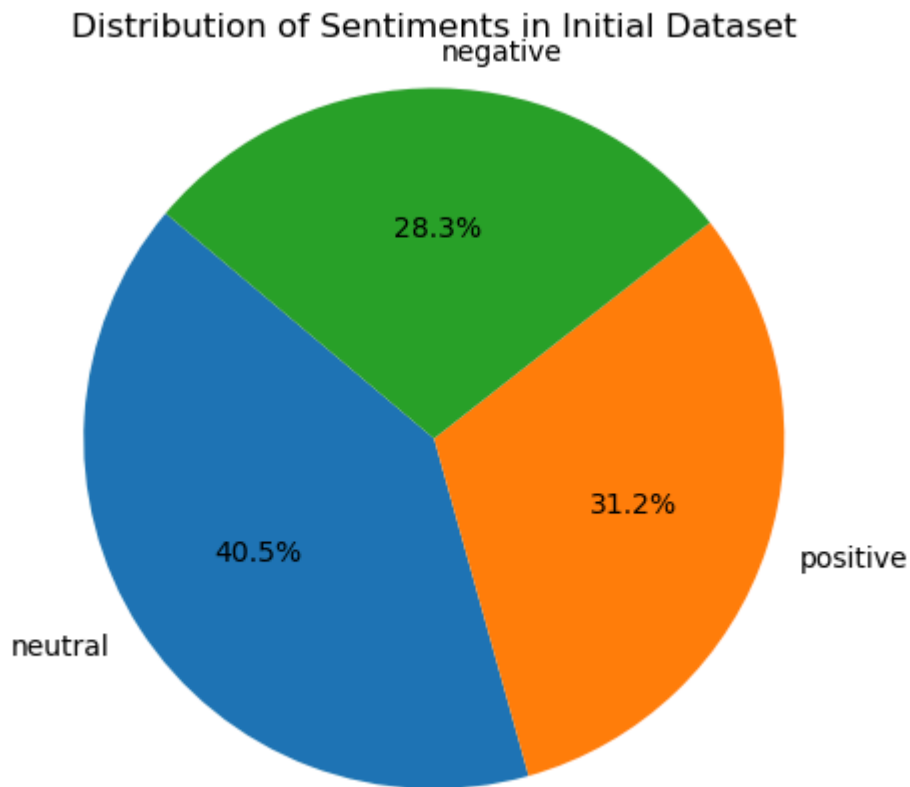
Classification Report for Sentiment Analyzer (VADER):

	precision	recall	f1-score	support
negative	0.70	0.61	0.65	1001
neutral	0.73	0.48	0.58	1430
positive	0.56	0.88	0.68	1103
accuracy			0.64	3534
macro avg	0.66	0.66	0.64	3534
weighted avg	0.67	0.64	0.63	3534

```
In [48]: # Count the occurrences of each sentiment label
sentiment_counts = df['sentiment'].value_counts()

# Plotting pie chart for initial dataset sentiments
plt.figure(figsize=(7, 5))
plt.pie(sentiment_counts, labels=sentiment_counts.index, autopct='%1.1f%%', star
plt.title('Distribution of Sentiments in Initial Dataset')
plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.

plt.show()
```



```
In [49]: import matplotlib.pyplot as plt
import pandas as pd

# Load the DataFrame with predictions
results_df = pd.read_csv('predictions.csv')

# Ensure that necessary columns are present
required_columns = ['SVM Predictions', 'Sentiment Analyzer Predictions']

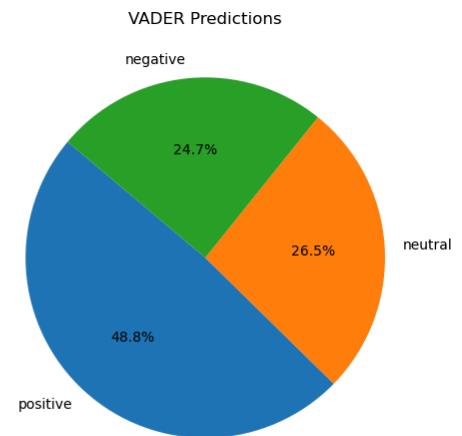
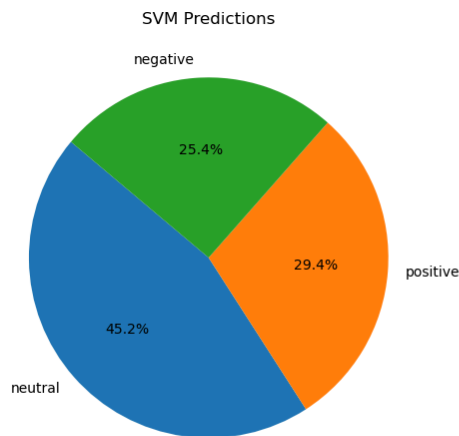
if not all(col in results_df.columns for col in required_columns):
    print("Required columns not found in DataFrame.")
else:
    svm_prediction_counts = results_df['SVM Predictions'].value_counts()
    vader_prediction_counts = results_df['Sentiment Analyzer Predictions'].value_counts()

    # Calculate percentages
    total_samples = len(results_df)
    svm_prediction_percentages = svm_prediction_counts / total_samples * 100
    vader_prediction_percentages = vader_prediction_counts / total_samples * 100

    # Plotting pie chart for SVM predictions
    plt.figure(figsize=(15, 5))
    plt.subplot(1, 2, 1)
    plt.pie(svm_prediction_counts, labels=svm_prediction_counts.index, autopct='%1.1f%%')
    plt.title('SVM Predictions')

    # Plotting pie chart for VADER predictions
    plt.subplot(1, 2, 2)
    plt.pie(vader_prediction_counts, labels=vader_prediction_counts.index, autopct='%1.1f%%')
    plt.title('VADER Predictions')

    plt.tight_layout()
    plt.show()
```

Step 9: User Input and Prediction

```
In [57]: user_input = input("Enter your text: ")
preprocessed_input = preprocess_text(user_input)

# Step 5: Make predictions using SVM classifier
svm_prediction = svm_classifier.predict([preprocessed_input])

# Step 6: Make predictions using SentimentIntensityAnalyzer (VADER)
sentiment_scores = sid.polarity_scores(preprocessed_input)

if sentiment_scores['compound'] >= 0.05:
    vader_prediction = 'positive'
elif sentiment_scores['compound'] <= -0.05:
    vader_prediction = 'negative'
else:
    vader_prediction = 'neutral'

print("\nUser Input: ", user_input)
print("\nPreprocessed Text: ", preprocessed_input)
print("\nPrediction using SVM classifier: ", svm_prediction)
print("\nPrediction using SentimentIntensityAnalyzer: ", vader_prediction)
```

User Input: Adorable in her innocence, yet victimized by life's cruelty, Loved deeply, yet often finds herself crying silently, A whirlwind of emotions—happy, sad, and fearful—she journeys bravely.

Preprocessed Text: adorable innocence yet victimized life cruelty loved deeply yet often find cry silently whirlwind emotions—happy sad fearful—she journey bravely

Prediction using SVM classifier: ['negative']

Prediction using SentimentIntensityAnalyzer: neutral

Step 9: Emotion Analysis

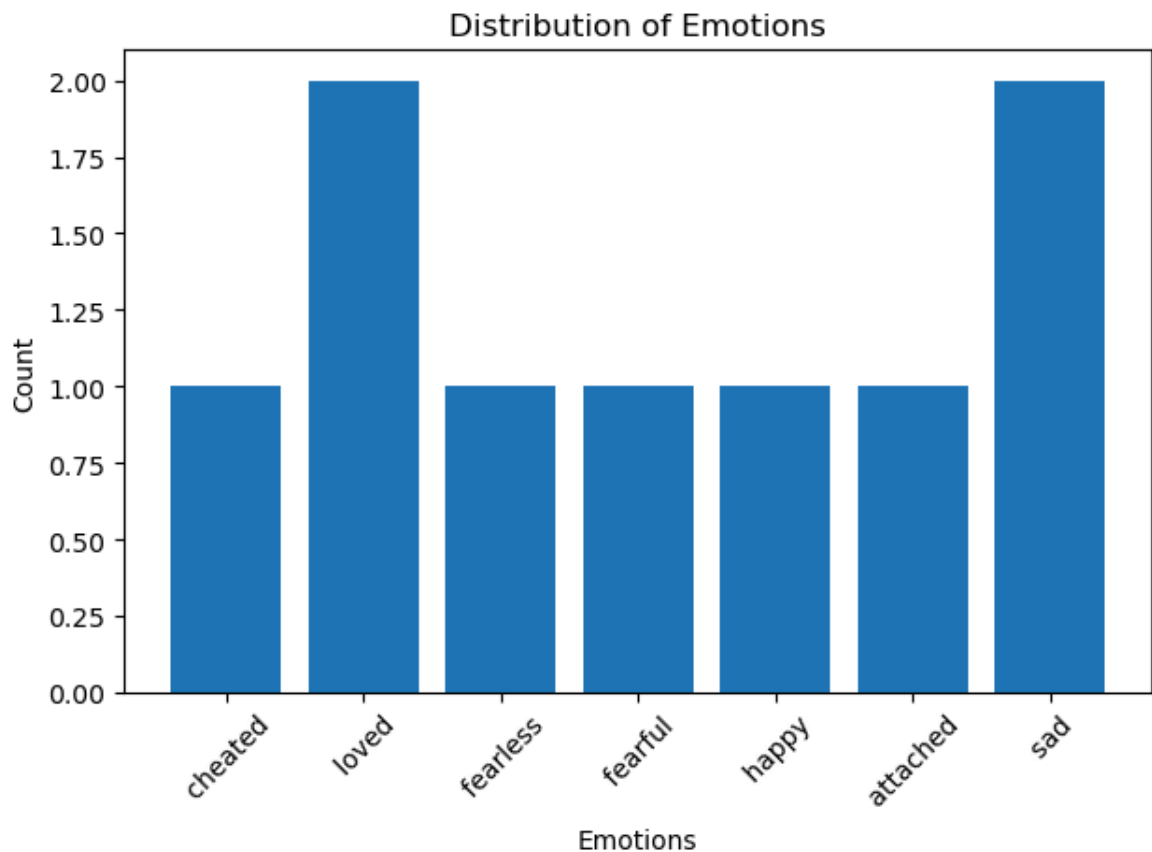
```
In [55]: from collections import Counter
```

```
In [58]: emotion_list = []
with open('emotions.txt', 'r') as file:
    for line in file:
        clear_line = line.replace("\n", '').replace(", ", '').replace("'", '').strip()
        word, emotion = clear_line.split(':')

        if word in preprocessed_input:
            emotion_list.append(emotion)

# Count the occurrences of each emotion
emotion_counter = Counter(emotion_list)

# Plot the bar graph
plt.bar(emotion_counter.keys(), emotion_counter.values())
plt.xlabel('Emotions')
plt.ylabel('Count')
plt.title('Distribution of Emotions')
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('emotions_bar_graph.png')
plt.show()
```



In []: